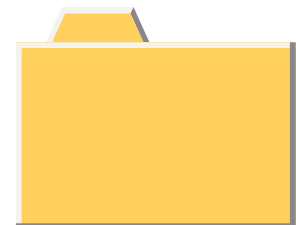
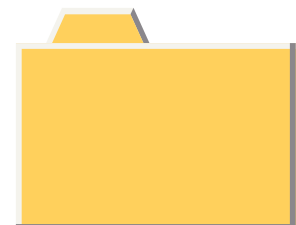


Pendahuluan



Program 01



Program 02



Program 03

PROJECT AKHIR

ALGORITMA PEMROGRAMAN

Dosen Pengampu: Lutfiah Maharani Siniwi, S.Stat., M.Stat.

Winny Anindita Natasya / K1D025045

Almass Rahma Shaliha / K1D025054

Yudalaga Satriaaji / K1D025057

Arrohmanasyah Al Hakim / K1D025073

STATISTIKA B

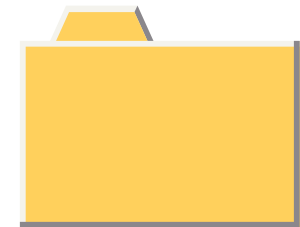
KELOMPOK E

Tahun Ajaran 2025/2026

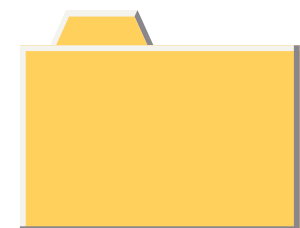


Okay

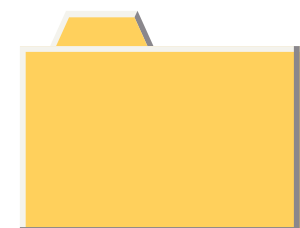




Pendahuluan



Program 01



Program 02



Program 03

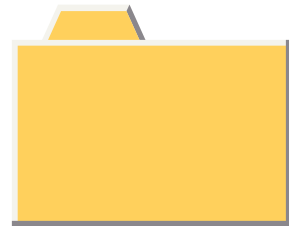
PENDAHULUAN

Di era perkembangan teknologi saat ini, kemampuan pemrograman menjadi salah satu keterampilan yang penting untuk dimiliki. Pemrograman tidak hanya digunakan untuk membuat aplikasi, tetapi juga untuk membantu menyelesaikan berbagai permasalahan secara sistematis dan efisien. Oleh karena itu, Project Algoritma dan Pemrograman ini dibuat untuk melatih mahasiswa dalam menganalisis masalah, menyusun algoritma, serta mengimplementasikan solusi melalui enam studi kasus menggunakan bahasa Python dan R. Melalui project ini, mahasiswa dapat menerapkan konsep-konsep dasar pemrograman yang telah dipelajari selama perkuliahan, mulai dari analisis masalah, penyusunan pseudocode dan flowchart, hingga implementasi dan pengujian program. Selain itu, project ini juga bertujuan untuk meningkatkan kemampuan berpikir logis, pemecahan masalah, serta keterampilan bekerja sama dalam tim.

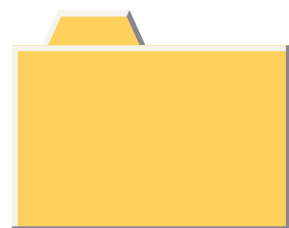


Okay

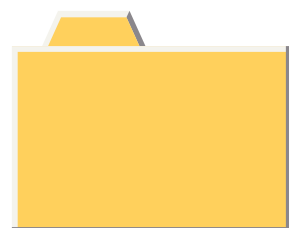




Program 01



Program 02



Program 03



Program 04

Program 1 - Analisis Teks

Deskripsi Masalah

Program ini dirancang untuk menganalisis sebuah teks dengan menghitung jumlah kata dan jumlah kalimat yang terdapat di dalamnya.

Logika Penyelesaian

- Pengguna memasukkan sebuah teks.
- Program menghitung jumlah tanda titik (.) untuk menentukan jumlah kalimat.
- Semua tanda titik dihapus dari teks agar tidak ikut dihitung sebagai kata.
- Teks dipisahkan menjadi beberapa kata berdasarkan spasi.
- Jumlah kata dihitung berdasarkan banyaknya elemen hasil pemisahan teks.
- Program menampilkan jumlah kata dan jumlah kalimat.

Contoh Hasil Eksekusi

```
> teks_tanpa_titik <- gsub("\\.", "", teks)
> kata <- unlist(strsplit(teks_tanpa_titik, "\\s+"))
> jumlah_kata <- length(kata)
> cat("Jumlah kata =", jumlah_kata, "\n")
Jumlah kata = 92
> cat("Jumlah kalimat =", jumlah_kalimat, "\n")
Jumlah kalimat = 5
```

Output RStudio

```
teks = input("Masukkan teks: ")
```

```
jumlah_kalimat = teks.count(".")
```

```
teks_tanpa_titik = teks.replace(".", "")
```

```
kata = teks_tanpa_titik.split()
```

```
jumlah_kata = len(kata)
```

```
print(f"Jumlah kata = {jumlah_kata}")
```

```
print(f"Jumlah kalimat = {jumlah_kalimat}")
```

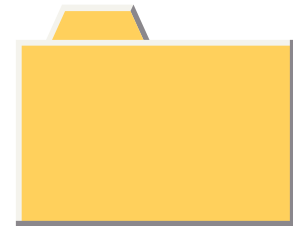
```
Masukkan teks: Media sosial atau disebut juga dengan jejaring sosi
Jumlah kata = 92
Jumlah kalimat = 5
```

Output Python

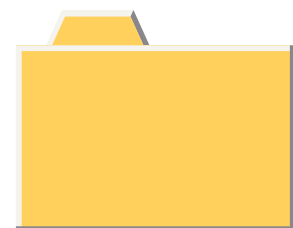


Okay

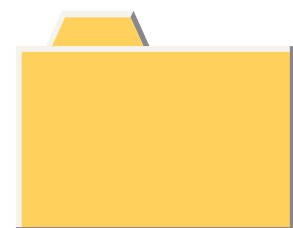




Program 02



Program 03



Program 04



Program 05

Program 2 - Rekayasa Objek String K1-K4

Deskripsi Masalah

Program ini dirancang untuk membuat dan menampilkan empat objek string (K1, K2, K3, dan K4) yang mengandung berbagai format karakter khusus, seperti tanda petik tunggal dan tanda petik ganda, sesuai dengan kalimat yang telah ditentukan.

Logika Penyelesaian

- Program mendefinisikan empat objek string yaitu K1, K2, K3, dan K4.
- Setiap string disimpan dengan format tanda petik yang sesuai agar dapat ditampilkan dengan benar.
- Karakter khusus seperti tanda petik diatur menggunakan penulisan string yang sesuai.
- Program menampilkan isi setiap objek string secara berurutan.
- Hasil output yang muncul harus sama dengan kalimat yang telah ditentukan pada soal.

Contoh Hasil Eksekusi

```
> # Cetak hasil menggunakan cat() agar tanda escape (\) hilang saat tampil
> cat(K1, "\n")
Saya tak 'kan menyerah.
> cat(K2, "\n")
Ia berkata, "Aku menyayangimu."
> cat(K3, "\n")
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

Output RStudio

```
# Program Membuat Objek String K1-K4 Sesuai Soal
K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
```

```
print(K1)
print(K2)
print(K3)
print(K4)
```

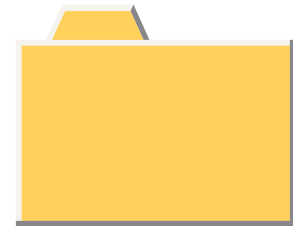
```
Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

Output Python

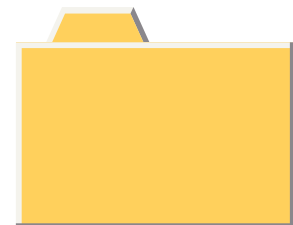


Okay





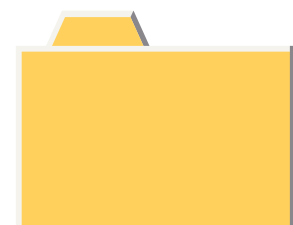
Program 03



Program 04



Program 05



Program 06

Program 3 - Akar Persamaan Kuadrat

Deskripsi Masalah

Program ini dirancang untuk menentukan akar-akar dari persamaan kuadrat $ax^2 + bx + c = 0$ berdasarkan tiga koefisien yang dimasukkan pengguna, dengan mempertimbangkan kemungkinan akar bertipe real maupun imajiner.

Logika Penyelesaian

- Nilai diskriminan ($D = b^2 - 4ac$) ditentukan terlebih dahulu sebagai dasar pengambilan keputusan
- $D > 0 \rightarrow$ persamaan memiliki dua akar real yang berbeda
- $D = 0 \rightarrow$ persamaan memiliki satu akar real kembar
- $D < 0 \rightarrow$ persamaan hanya memiliki akar imajiner, sehingga program menampilkan informasi tersebut tanpa menghitung nilai akarnya
- Akar real ditampilkan dalam format tiga angka desimal sesuai ketentuan soal

Contoh Hasil Eksekusi

```
=== Program Pencari Akar Persamaan Kuadrat ===  
Bentuk umum:  $ax^2 + bx + c = 0$   
Masukkan koefisien a: 1  
Masukkan koefisien b: -5  
Masukkan koefisien c: 6
```

```
Persamaan memiliki dua akar real yang berbeda:  
x1 = 3.000  
x2 = 2.000
```

Output RStudio

```
=== Program Pencari Akar Persamaan Kuadrat ===  
Bentuk umum:  $ax^2 + bx + c = 0$   
Masukkan koefisien a: 1  
Masukkan koefisien b: 2  
Masukkan koefisien c: 5
```

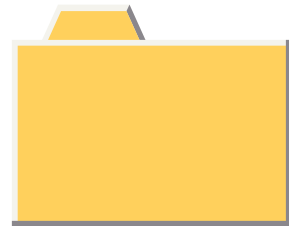
```
Persamaan tersebut hanya memiliki akar-akar imajiner.
```

Output Python

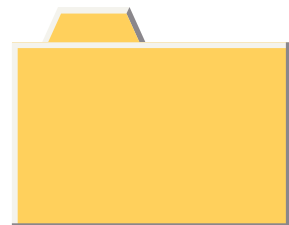


Okay





Program 04



Program 05



Program 06



.py & .R

Program 4 - Pengolahan NIP ASN

Deskripsi Masalah

Program ini dirancang untuk menentukan tanggal lahir seorang ASN berdasarkan data NIP yang diberikan. Delapan digit pertama pada NIP digunakan untuk memperoleh informasi tahun lahir, bulan lahir, dan tanggal lahir ASN.

Logika Penyelesaian

- NIP diperiksa untuk memastikan terdiri dari minimal delapan digit dan hanya berisi angka.
- Delapan digit pertama dipisahkan menjadi tahun, bulan, dan tanggal lahir.
- Kode bulan diubah menjadi nama bulan menggunakan struktur percabangan.
- Validasi dilakukan untuk memastikan kode bulan sesuai dengan data yang tersedia.
- Tanggal lahir atau pesan kesalahan ditampilkan sebagai hasil akhir.

Contoh Hasil Eksekusi

```
+ if (bulan != "Tidak Valid") {  
+   # Perubahan di sini: Menambahkan teks NIP dan variabel ni  
+   cat(paste("NIP:", nip, "\nTanggal Lahir:", tanggal, bulan  
+ } else {  
+   cat("Error: Format bulan pada NIP tidak valid.\n")  
+ }  
+ } else {  
+   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")  
+ }
```

NIP: 199508172023011001
Tanggal Lahir: 17 Agustus 1995

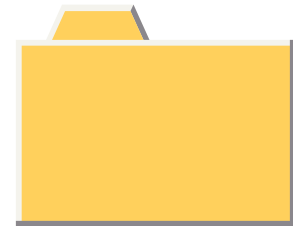
Output RStudio

```
if bulan != "Tidak Valid":  
    # Menampilkan NIP dan Tanggal Lahir  
    print(f"NIP: {nip}")  
    print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")  
else:  
    print("Error: Format bulan pada NIP tidak valid.")  
else:  
    print("Error: NIP harus berupa angka dan minimal 8 digit.")
```

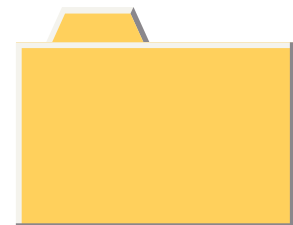
NIP: 199508172023011001
Tanggal Lahir: 17 Agustus 1995

Output Python

Okay



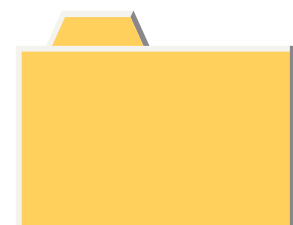
Program 05



Program 06



.py & .R



Uji Program

Program 5 - Klasifikasi Cluster 3 Dimensi

Program ini dirancang untuk menentukan cluster suatu titik pada ruang tiga dimensi berdasarkan jarak terdekat terhadap pusat cluster A, B, dan C.

Logika Penyelesaian

- Titik pusat Cluster A, B, dan C ditentukan sebagai acuan dalam proses pengelompokan titik U.
- Koordinat titik U diperiksa untuk memastikan nilai yang dimasukkan merupakan data yang valid.
- Jarak antara titik U dengan masing-masing pusat cluster dihitung menggunakan rumus jarak Euclidean tiga dimensi.
- Nilai jarak dari ketiga pusat cluster dibandingkan untuk menentukan jarak yang paling kecil.
- Titik U digolongkan ke dalam Cluster A, B, atau C berdasarkan jarak terdekat.
- Hasil perhitungan jarak serta hasil klasifikasi cluster ditampilkan sebagai output program.

Contoh Hasil Eksekusi

```
+ } else {  
+   cat("Error: Input koordinat harus berupa angka!\n")  
+ }
```

```
Jarak titik U ke Pusat A : 5.000  
Jarak titik U ke Pusat B : 1.414  
Jarak titik U ke Pusat C : 10.770  
>> Titik U digolongkan sebagai anggota: Cluster B
```

Output RStudio

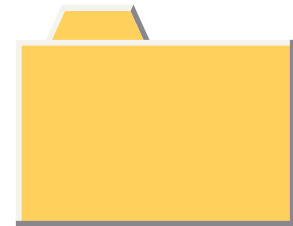
```
--- Klasifikasi Cluster 3 Dimensi ---
```

```
Koordinat Titik U yang Diuji -> x: 2, y: -3, z: 6
```

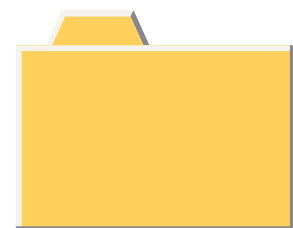
```
Jarak titik U ke Pusat A : 5.000  
Jarak titik U ke Pusat B : 1.414  
Jarak titik U ke Pusat C : 10.770  
>> Titik U digolongkan sebagai anggota: Cluster B
```

Output Python

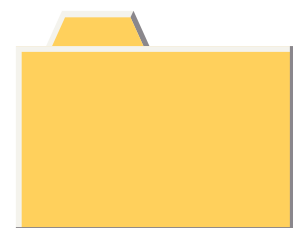
Okay



Program 06



.py & .R



Uji Program



Penutup

Program 6 - Interval Konfidensi Proporsi

Program ini dirancang untuk menghitung interval konfidensi proporsi populasi berdasarkan proporsi sampel ($\hat{p}$) dan tingkat kepercayaan (α) yang dimasukkan pengguna.

Logika Penyelesaian

- Nilai z ditentukan berdasarkan α yang dimasukkan: 1,645 jika $\alpha = 10\%$, dan 1,96 jika $\alpha = 5\%$
- Batas bawah dan batas atas interval dihitung menggunakan rumus $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$
- Validasi input dilakukan terlebih dahulu: program menampilkan pesan kesalahan jika $\hat{p}$ bernilai < 0 atau > 1
- Perhitungan dibungkus dalam fungsi agar program lebih modular dan dapat dipanggil ulang
- Hasil interval ditampilkan dalam format yang mudah dibaca (batas bawah $< p <$ batas atas)

Contoh Hasil Eksekusi

```
> # Input
> p_hat <- as.numeric(readline("Masukkan nilai proporsi sampel (p_hat): "))
Masukkan nilai proporsi sampel (p_hat): 0.4
> n <- as.integer(readline("Masukkan nilai n: "))
Masukkan nilai n: 100
> alpha <- as.numeric(readline("Masukkan nilai alpha (0.10 atau 0.05): "))
Masukkan nilai alpha (0.10 atau 0.05): 0.05
> interval_kepercayaan(p_hat, n, alpha)
```

Interval Kepercayaan 95%
0.3040 < p < 0.4960

Output RStudio

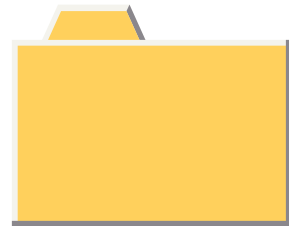
```
# Input
p_hat = float(input("Masukkan proporsi sampel (p_hat): "))
n = int(input("Masukkan ukuran sampel (n): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi proporsi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi harus berada antara 0 dan 1.")
else:
    hitung_interval(p_hat, n, alpha)
```

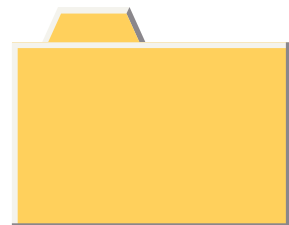
```
Masukkan proporsi sampel (p_hat): 1.2
Masukkan ukuran sampel (n): 100
Masukkan nilai alpha (0.10 atau 0.05): 0.05
Error: Proporsi harus berada antara 0 dan 1.
```

Output Python

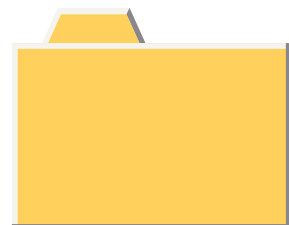
Okay



.py & .R



Uji Program



Penutup



End

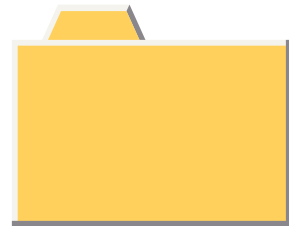
PERBANDINGAN PYTHON DAN R

Aspek Perbandingan	Python	R
Sistem Indeksasi (Contoh: NIP ASN)	Dimulai dari 0 (zero-based). Contoh pemotongan: nip[0:4]	Dimulai dari 1 (one-based). Contoh pemotongan: substr(nip, 1, 4)
Manipulasi Teks (Contoh: Hitung Kata)	Sangat praktis dengan method bawaan string. Contoh: .split() otomatis memisah kata.	Output berupa list, butuh penyesuaian. Contoh: strsplit() butuh unlist() untuk dihitung panjangnya.
Perhitungan Matematis (Contoh: Jarak Cluster)	Membutuhkan impor modul tambahan. Contoh: Harus import math untuk pakai math.sqrt().	Sangat native dan efisien untuk rumus. Contoh: Bisa langsung pakai sqrt() tanpa library tambahan.
Struktur Percabangan (Contoh: Cek Akar Imajiner)	Mengandalkan indentasi (spasi/tab) untuk membatasi blok <i>if-elif-else</i> .	Menggunakan kurung kurawal {} untuk membatasi blok <i>if-else if-else</i> .

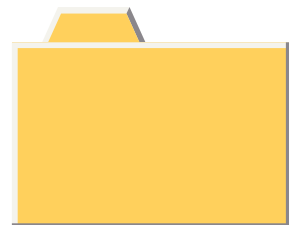


Okay

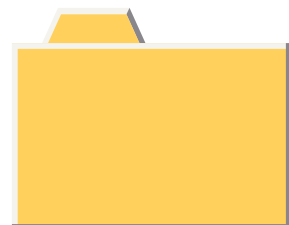




.py & .R



Uji Program



Penutup



End

PENGUJIAN PROGRAM

1. Pengujian Kasus Normal (*Normal Case*)

Tujuan: Memastikan program menghasilkan output yang benar ketika diberikan input yang valid dan ideal.

Contoh: Memasukkan teks standar pada Program 1, atau memasukkan koefisien persamaan kuadrat yang menghasilkan akar real berbeda pada Program 3 ($D > 0$).

2. Pengujian Kondisi Batas (*Boundary Case*)

Tujuan: Menguji sensitivitas program saat menerima input yang berada tepat di batas ketentuan.

Contoh: Memasukkan nilai proporsi sampel tepat di angka 0 atau 1 pada Program 6 untuk melihat apakah batas interval konfidensi tetap terhitung secara logis.

3. Pengujian Kondisi Khusus / Input Tidak Valid (*Error Handling*)

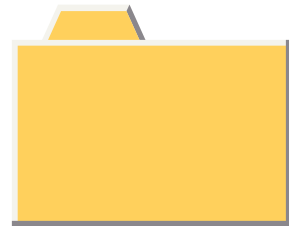
Tujuan: Memastikan program memiliki error handling yang baik dan tidak langsung crash ketika pengguna memasukkan data yang salah.

Contoh: Memasukkan nilai proporsi > 1 atau < 0 pada Program 6 (program akan memunculkan pesan kesalahan), atau mendeteksi nilai diskriminan negatif ($D < 0$) pada Program 3 untuk langsung mengategorikannya sebagai akar imajiner tanpa memaksa menghitung nilai akarnya.

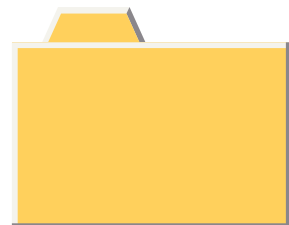


Okay





.py & .R



Uji Program



Penutup



End

KESIMPULAN

Kelompok kami berhasil mengimplementasikan 6 studi kasus komputasional menggunakan bahasa R dan Python dengan hasil logika yang setara dan konsisten.

Kelebihan Python: Terbukti sangat efisien dan praktis dalam melakukan manipulasi data string serta analisis teks (Program 1 & 2).

Kelebihan R: Sangat *native*, cepat, dan intuitif ketika mengeksekusi perhitungan formula statistik maupun matematika (Program 5 & 6).

KENDALA

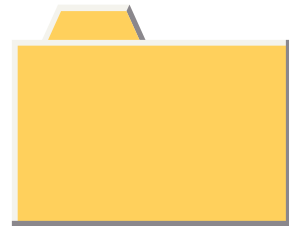
Di sisi lain, kelompok kami menghadapi beberapa kendala teknis selama proses pengembangan.

Tantangan utama terletak pada ketelitian menyelaraskan sintaks akibat perbedaan mendasar kedua bahasa, seperti sistem indeksasi Python yang dimulai dari nol dan R yang dimulai dari satu, serta pengamanan logika error handling agar program tidak crash saat menerima input tidak valid.

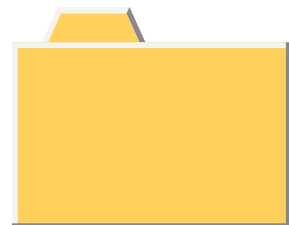


Okay

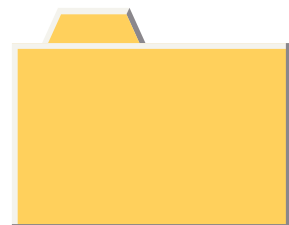




.py & .R



Uji Program



Penutup



End

thAnK you ;)

GitHub Repository: <https://github.com/bj1aro/project-alpro-kelompok-E-Stats-B>



Okay

